

Towards Closed-loop Deep Brain Stimulation via Optimal Control

Malvern Madondo¹

Deepanshu Verma² Lars Ruthotto² Nicholas Au Yong³

Department of: Computer Science¹, Mathematics², Neurosurgery³

SIAM MDS22 - Deep Learning and Optimal Control in Data Space

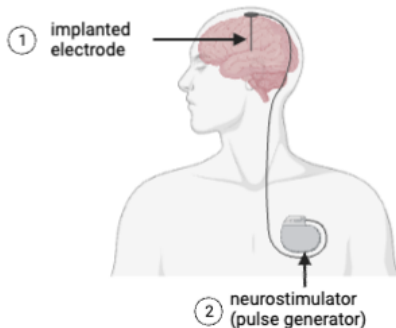
Funding Acknowledgements: Google PhD Fellowship in Computational Neural and Cognitive Sciences, AFOSR: FA9550-20-1-0372 and DOE RISE: ASCR 20-023231



EMORY
UNIVERSITY

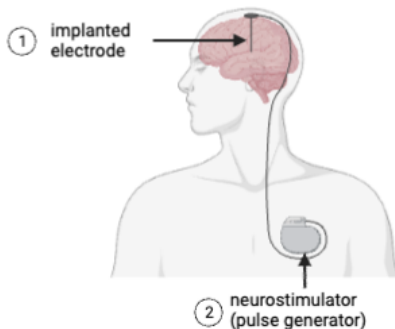
Deep Brain Stimulation (DBS)

- Delivers electrical pulses to the brain via surgically implanted electrodes.
- Used for medication resistant neurological disorders e.g., Parkinson's Disease.

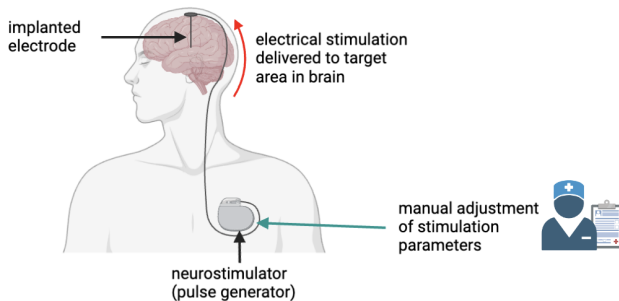


Deep Brain Stimulation (DBS)

- Delivers electrical pulses to the brain via surgically implanted electrodes.
- Used for medication resistant neurological disorders e.g., Parkinson's Disease.

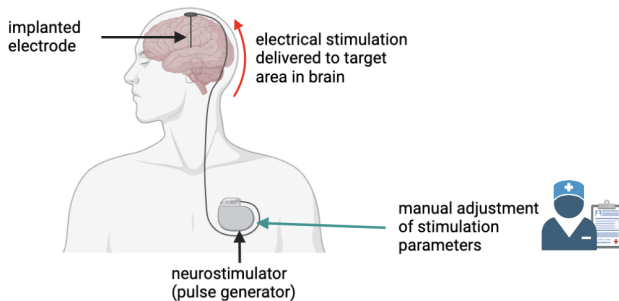


Clinical Goal: **Find optimal stimulation to manage symptoms**



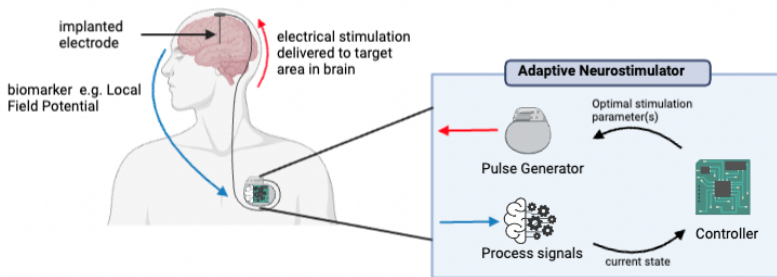
Overview

- 1 Clinicians **manually program** the neurostimulator (often via trial-and-error)



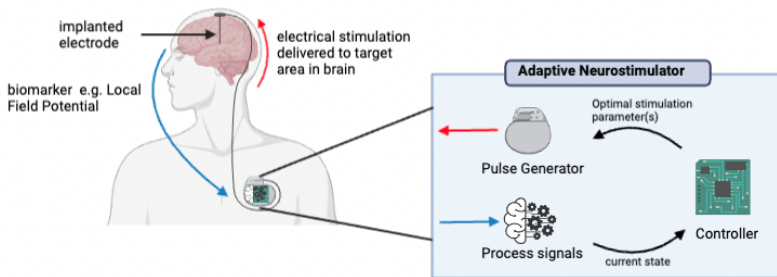
Overview

- 1 Clinicians **manually program** the neurostimulator (often via trial-and-error)
- 2 **Continuously** deliver a **fixed** amount of electrical impulse to patient.



Overview

- 1 Controller adjusts the stimulation based on **feedback**

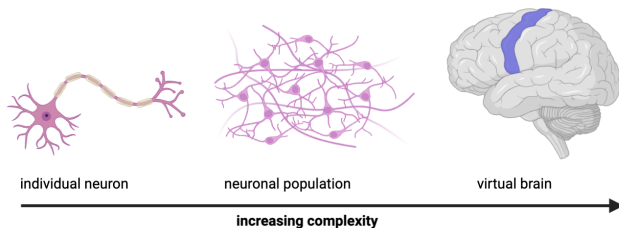


Overview

- 1 Controller adjusts the stimulation based on **feedback**
- 2 **Adaptive**: only deliver electrical stimulation when needed

A rich application area for Deep learning and Optimal Control. Possible research directions:

- ① Modeling dynamics for a single neuron to large neuronal populations
- ② Characterizing neurostimulation as a control problem
- ③ Applying control-based approaches to solve the problem
 - Combine Deep Learning and Optimal Control



Consider neuronal dynamics given by

$$\frac{dz}{dt}(t) = f(t, z(t)) + e_1 u(t), \quad t \in [0, T], \quad e_1 = [1, 0, 0, 0]^\top,$$

with initial data $z(0) = x$ where

- $z(t) \in \mathbb{R}^4$ denotes the **state** variables (i.e., membrane potential z_0 and ion channel gating variables: z_1, z_2, z_3)
- $u(t)$ is the **control** variable (external current provided as input at time t).
- f describes time evolution of z , e.g., based on Hodgkin and Huxley [1952]

Consider neuronal dynamics given by

$$\frac{dz}{dt}(t) = f(t, z(t)) + e_1 u(t), \quad t \in [0, T], \quad e_1 = [1, 0, 0, 0]^T,$$

with initial data $z(0) = x$ where

- $z(t) \in \mathbb{R}^4$ denotes the **state** variables (i.e., membrane potential z_0 and ion channel gating variables: z_1, z_2, z_3)
- $u(t)$ is the **control** variable (external current provided as input at time t).
- f describes time evolution of z , e.g., based on Hodgkin and Huxley [1952]

$$f(t, z(t)) = \begin{pmatrix} -(I_{\text{Na}}(t, z_0(t)) + I_{\text{K}}(t, z_0(t)) + I_{\text{L}}(t, z_0(t))) \\ \alpha_m(z_0(t))(1 - z_1(t)) - \beta_m(z_0(t))z_1(t) \\ \alpha_n(z_0(t))(1 - z_2(t)) - \beta_n(z_0(t))z_2(t) \\ \alpha_h(z_0(t))(1 - z_3(t)) - \beta_h(z_0(t))z_3(t) \end{pmatrix}.$$

where

- $I_{\text{Na}}, I_{\text{K}}, I_{\text{L}}$ are sodium, potassium, and leak ion channel currents
- α_x and β_x are voltage-dependent rate variables with $x \in \{m, n, h\}$

Objective

Find an optimal control $u^*(t)$, $0 \leq t \leq T$ that **minimizes** the **value function**:

$$\Phi(t, z(t)) = \inf_u \left(\int_t^T L(s, z(s), u(s)) ds + G(z(T)) \right),$$

where

- $L(t, z, u)$ is the running cost (Lagrangian cost)
- $G(z(T))$ is the terminal cost

Methods for finding optimal solution:

- ① local: generates a control policy based on a fixed initial state $z(0)$.
- ② global: find OC policy for any given state $z(t)$, $t \in [0, T]$

- Pontryagin's Maximum Principle provides necessary conditions for $u^*(t)$
- **Optimality conditions** for $t < s \leq T$ are given by

$$\begin{cases} \partial_t p(s) = \nabla_z \mathcal{H}(s, z^*(s), p(s), u^*(s)), \\ \partial_s z^*(s) = -\nabla_p \mathcal{H}(s, z^*(s), p(s), u^*(s)) \\ z^*(0) = z, \quad p(T) = \nabla_z G(z^*(T)) \end{cases}$$

where

- $\mathcal{H}$ is the Hamiltonian of the system, defined as

$$\mathcal{H}(t, z, p, u) = -L(t, z(t), u(t)) - p^\top \cdot [f(z(t), t) + e_1 u(t)]$$

- p is the costate/adjoint variable.

Then $u^*(s) \in \arg \max_u \mathcal{H}(s, z^*(s), p(s), u(s))$

- Provides necessary and sufficient conditions for optimality.
- At optimality, Φ **satisfies the HJB equation**:

$$\begin{cases} -\partial_t \Phi(t, z(t)) + \sup_{u \in U} \mathcal{H}(t, z, \nabla_z \Phi(t, z(t)), u(t)) = 0, \\ \Phi(T, z(T)) = G(z(T)), \end{cases}$$

- The optimal control u^* can be recovered online in **feedback form** as

$$u^*(s) \in \arg \max_u \mathcal{H}(s, z^*(s), \nabla_z \Phi(s, z^*(s)), u(s))$$

Note: In HJB, optimal controls depend on $\nabla_z \Phi$, i.e.,

$$p(s) = \nabla_z \Phi(s, z(s)), \quad t < s \leq T.$$

NeuralHJB

- Combines PMP and HJB approaches while parameterizing Φ with a neural network as per Ruthotto et al. [2020] and Onken et al. [2021].
- **Today @ 3:15 PM in Palm Room 6** **Advances and Challenges in Solving HJB Equations Arising in Optimal Control.** Deepanshu Verma [MS65 Part II]



JOSS
The Journal of Open Source Software

hessQuik: Fast Hessian computation of composite functions

Elizabeth Newman¹

¹ Envisy, University of

hessQuik is a lightweight (in terms of composite function) and fast (in terms of Hessian computation) tool for computing Hessians of composite functions. The code is easy to use, and the results are easy to interpret. The results are easy to interpret.

Editor: Matthew Hahn, Samuel

JOSS, v10.1238 (2023-06-06)

Software

- Review of
- Review of
- Review of

Editor: Matthew Hahn, Samuel

Reviewers:

A Neural Network Approach for High-Dimensional Optimal Control Applied to Multi-Agent Path Finding

Drake O'Neil, Leonor Nurbayeva, Xinling Li, Gary W. Fung, Stanley Osher, and Luis Ruffatto

Abstract

We propose a neural network approach that solves the problem of finding optimal control policies and demonstrates its effectiveness on a variety of problems. We apply this approach to a multi-agent path finding problem, where the policy must be given to a neural network. Specifically, we have a neural network (NN) and a Python-based reinforcement learning (RL) algorithm for approximating the optimal control policy. We demonstrate that the NN can learn approximately optimal controls to maximize reward while avoiding obstacles and avoiding collisions. Once the policy function is learned, generating a control at a given position requires little computation. In contrast, different control policies can be generated by changing the input to the NN. In summary, we aim to use this software to solve the optimal control problem and provide a tool for the user to solve the RL problem. Therefore, we provide a tool for the user to solve the RL problem.

Received: OCT 2, 2023



JOSS
The Journal of Open Source Software

A machine learning framework for solving high-dimensional mean field game and mean field control problems

Luis Ruffatto^{1,2}, Stanley J. Osher³, Leonor Nurbayeva⁴, and Gary W. Fung⁵

¹Department of Mathematics, Brown University, Providence, RI, 02912; ²Department of Mathematics, Brown University, Providence, RI, 02912; ³Department of Mathematics, University of California, Los Angeles, CA, 90095; ⁴Department of Mathematics, University of California, Los Angeles, CA, 90095; ⁵Department of Mathematics, University of California, Los Angeles, CA, 90095

Abstract

Mean field games (MFG) and mean field control (MFC) are critical areas of multistage problems for the efficient analysis of massive populations of interacting agents. Their uses of approximation and numerical methods, game theory, stochastic control, and machine learning, have been widely studied in the literature. In this paper, we provide a machine learning framework for the numerical solution of general MFG and MFC problems. State-of-the-art numerical methods for solving such problems often require discretization that leads to a curse of dimensionality. We approximate using high-dimensional problems by combining deep learning and reinforcement learning to solve these problems. We demonstrate that the NN can learn approximately optimal controls to maximize reward while avoiding obstacles and avoiding collisions. Once the policy function is learned, generating a control at a given position requires little computation. In contrast, different control policies can be generated by changing the input to the NN. In summary, we aim to use this software to solve the optimal control problem and provide a tool for the user to solve the RL problem. Therefore, we provide a tool for the user to solve the RL problem.

Received: OCT 2, 2023



JOSS
The Journal of Open Source Software

hessQuik: Fast Hessian computation of composite functions

Elizabeth Newman¹

¹ Envisy, University of

hessQuik is a lightweight (in terms of composite function) and fast (in terms of Hessian computation) tool for computing Hessians of composite functions. The code is easy to use, and the results are easy to interpret. The results are easy to interpret.

Editor: Matthew Hahn, Samuel

JOSS, v10.1238 (2023-06-06)

Software

- Review of
- Review of
- Review of

Editor: Matthew Hahn, Samuel

Reviewers:

A Neural Network Approach for High-Dimensional Optimal Control Applied to Multi-Agent Path Finding

Drake O'Neil, Leonor Nurbayeva, Xinling Li, Gary W. Fung, Stanley Osher, and Luis Ruffatto

Abstract

We propose a neural network approach that solves the problem of finding optimal control policies and demonstrates its effectiveness on a variety of problems. We apply this approach to a multi-agent path finding problem, where the policy must be given to a neural network. Specifically, we have a neural network (NN) and a Python-based reinforcement learning (RL) algorithm for approximating the optimal control policy. We demonstrate that the NN can learn approximately optimal controls to maximize reward while avoiding obstacles and avoiding collisions. Once the policy function is learned, generating a control at a given position requires little computation. In contrast, different control policies can be generated by changing the input to the NN. In summary, we aim to use this software to solve the optimal control problem and provide a tool for the user to solve the RL problem. Therefore, we provide a tool for the user to solve the RL problem.

Received: OCT 2, 2023



JOSS
The Journal of Open Source Software

A machine learning framework for solving high-dimensional mean field game and mean field control problems

Luis Ruffatto^{1,2}, Stanley J. Osher³, Leonor Nurbayeva⁴, and Gary W. Fung⁵

¹Department of Mathematics, Brown University, Providence, RI, 02912; ²Department of Mathematics, Brown University, Providence, RI, 02912; ³Department of Mathematics, University of California, Los Angeles, CA, 90095; ⁴Department of Mathematics, University of California, Los Angeles, CA, 90095; ⁵Department of Mathematics, University of California, Los Angeles, CA, 90095

Abstract

Mean field games (MFG) and mean field control (MFC) are critical areas of multistage problems for the efficient analysis of massive populations of interacting agents. Their uses of approximation and numerical methods, game theory, stochastic control, and machine learning, have been widely studied in the literature. In this paper, we provide a machine learning framework for the numerical solution of general MFG and MFC problems. State-of-the-art numerical methods for solving such problems often require discretization that leads to a curse of dimensionality. We approximate using high-dimensional problems by combining deep learning and reinforcement learning to solve these problems. We demonstrate that the NN can learn approximately optimal controls to maximize reward while avoiding obstacles and avoiding collisions. Once the policy function is learned, generating a control at a given position requires little computation. In contrast, different control policies can be generated by changing the input to the NN. In summary, we aim to use this software to solve the optimal control problem and provide a tool for the user to solve the RL problem. Therefore, we provide a tool for the user to solve the RL problem.

Received: OCT 2, 2023



JOSS
The Journal of Open Source Software

hessQuik: Fast Hessian computation of composite functions

Elizabeth Newman¹

¹ Envisy, University of

hessQuik is a lightweight (in terms of composite function) and fast (in terms of Hessian computation) tool for computing Hessians of composite functions. The code is easy to use, and the results are easy to interpret. The results are easy to interpret.

Editor: Matthew Hahn, Samuel

JOSS, v10.1238 (2023-06-06)

Software

- Review of
- Review of
- Review of

Editor: Matthew Hahn, Samuel

Reviewers:

A Neural Network Approach for High-Dimensional Optimal Control Applied to Multi-Agent Path Finding

Drake O'Neil, Leonor Nurbayeva, Xinling Li, Gary W. Fung, Stanley Osher, and Luis Ruffatto

Abstract

We propose a neural network approach that solves the problem of finding optimal control policies and demonstrates its effectiveness on a variety of problems. We apply this approach to a multi-agent path finding problem, where the policy must be given to a neural network. Specifically, we have a neural network (NN) and a Python-based reinforcement learning (RL) algorithm for approximating the optimal control policy. We demonstrate that the NN can learn approximately optimal controls to maximize reward while avoiding obstacles and avoiding collisions. Once the policy function is learned, generating a control at a given position requires little computation. In contrast, different control policies can be generated by changing the input to the NN. In summary, we aim to use this software to solve the optimal control problem and provide a tool for the user to solve the RL problem. Therefore, we provide a tool for the user to solve the RL problem.

Received: OCT 2, 2023



JOSS
The Journal of Open Source Software

A machine learning framework for solving high-dimensional mean field game and mean field control problems

Luis Ruffatto^{1,2}, Stanley J. Osher³, Leonor Nurbayeva⁴, and Gary W. Fung⁵

¹Department of Mathematics, Brown University, Providence, RI, 02912; ²Department of Mathematics, Brown University, Providence, RI, 02912; ³Department of Mathematics, University of California, Los Angeles, CA, 90095; ⁴Department of Mathematics, University of California, Los Angeles, CA, 90095; ⁵Department of Mathematics, University of California, Los Angeles, CA, 90095

Abstract

Mean field games (MFG) and mean field control (MFC) are critical areas of multistage problems for the efficient analysis of massive populations of interacting agents. Their uses of approximation and numerical methods, game theory, stochastic control, and machine learning, have been widely studied in the literature. In this paper, we provide a machine learning framework for the numerical solution of general MFG and MFC problems. State-of-the-art numerical methods for solving such problems often require discretization that leads to a curse of dimensionality. We approximate using high-dimensional problems by combining deep learning and reinforcement learning to solve these problems. We demonstrate that the NN can learn approximately optimal controls to maximize reward while avoiding obstacles and avoiding collisions. Once the policy function is learned, generating a control at a given position requires little computation. In contrast, different control policies can be generated by changing the input to the NN. In summary, we aim to use this software to solve the optimal control problem and provide a tool for the user to solve the RL problem. Therefore, we provide a tool for the user to solve the RL problem.

Received: OCT 2, 2023



JOSS
The Journal of Open Source Software

hessQuik: Fast Hessian computation of composite functions

Elizabeth Newman¹

¹ Envisy, University of

hessQuik is a lightweight (in terms of composite function) and fast (in terms of Hessian computation) tool for computing Hessians of composite functions. The code is easy to use, and the results are easy to interpret. The results are easy to interpret.

Editor: Matthew Hahn, Samuel

JOSS, v10.1238 (2023-06-06)

Software

- Review of
- Review of
- Review of

Editor: Matthew Hahn, Samuel

Reviewers:

A Neural Network Approach for High-Dimensional Optimal Control Applied to Multi-Agent Path Finding

Drake O'Neil, Leonor Nurbayeva, Xinling Li, Gary W. Fung, Stanley Osher, and Luis Ruffatto

Abstract

We propose a neural network approach that solves the problem of finding optimal control policies and demonstrates its effectiveness on a variety of problems. We apply this approach to a multi-agent path finding problem, where the policy must be given to a neural network. Specifically, we have a neural network (NN) and a Python-based reinforcement learning (RL) algorithm for approximating the optimal control policy. We demonstrate that the NN can learn approximately optimal controls to maximize reward while avoiding obstacles and avoiding collisions. Once the policy function is learned, generating a control at a given position requires little computation. In contrast, different control policies can be generated by changing the input to the NN. In summary, we aim to use this software to solve the optimal control problem and provide a tool for the user to solve the RL problem. Therefore, we provide a tool for the user to solve the RL problem.

Received: OCT 2, 2023



JOSS
The Journal of Open Source Software

A machine learning framework for solving high-dimensional mean field game and mean field control problems

Luis Ruffatto^{1,2}, Stanley J. Osher³, Leonor Nurbayeva⁴, and Gary W. Fung⁵

¹Department of Mathematics, Brown University, Providence, RI, 02912; ²Department of Mathematics, Brown University, Providence, RI, 02912; ³Department of Mathematics, University of California, Los Angeles, CA, 90095; ⁴Department of Mathematics,

Approximate Φ using a neural network parameterized by θ , denoted by Φ_θ

$$\Phi_\theta(x) = w^\top N(x; \theta) + \frac{1}{2}x^\top (A^\top A)x + b^\top x + c$$

for space-time inputs $x = (t, z)$, trainable weights $\theta = (w, A, b, c)$, and a residual neural network $N(x; \theta)$.

Objective

$$\min_{\theta} \mathbb{E}_z \mathbb{E}_{x \sim \rho} \{ \ell(T) + g(z(T)) + \beta_1 c_{\text{HJ}}(T) + \beta_2 |\Phi_\theta(T, z(T)) - g(z(T))| \}$$

subject to

$$\partial_s \begin{pmatrix} z \\ \ell(s) \\ c_{\text{HJ}}(s) \end{pmatrix} = \begin{pmatrix} -\nabla_p H(s, z, \nabla_z \Phi(s, z)) \\ L(s, z, u^*) \\ | -\partial_t \Phi(s, z(s)) + H(s, z, \nabla_z \Phi(s, z, u^*)) | \end{pmatrix},$$

for $z(0) = x$, $\ell(0) = 0$, and $c_{\text{HJ}}(0) = 0$,

where β_1 and β_2 are hyperparameters, c_{HJ} is a penalty term derived from HJB

An Example: Target Potential $-20mV$

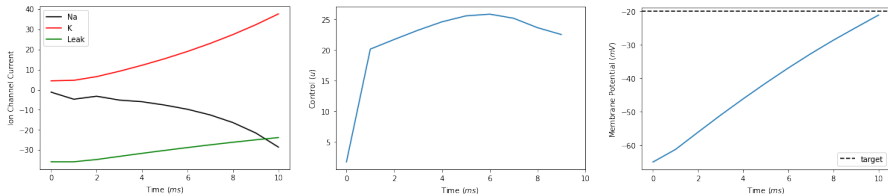


Figure: Observed ion channel currents, controls, and membrane potential from a local solution method

An Example: Target Potential $-20mV$

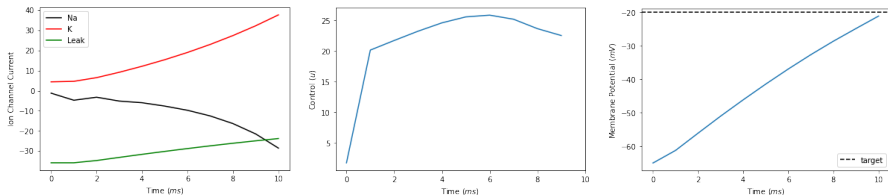


Figure: Observed ion channel currents, controls, and membrane potential from a local solution method

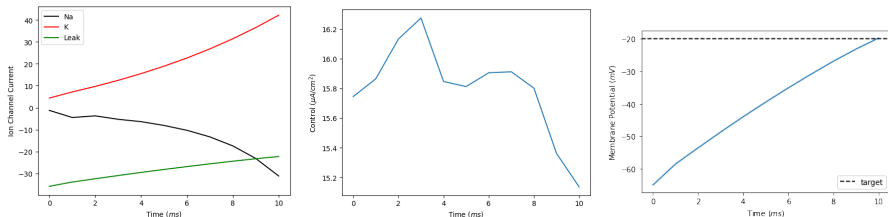


Figure: Observed ion channel currents, controls, and membrane potential from semi-global solution method

Beyond Single Neuron Dynamics

- Extension to large neuronal populations and network models
- Neural mass models approximate large-scale dynamics
- Opportunities to apply mean field theory and methods of finding approximate solutions to high-dimensional problems

Beyond Single Neuron Dynamics

- Extension to large neuronal populations and network models
- Neural mass models approximate large-scale dynamics
- Opportunities to apply mean field theory and methods of finding approximate solutions to high-dimensional problems

Mitigating Curse of Dimensionality

- CoD arises as neuronal models increase in complexity
- Solving HJB equations in higher dimensions is difficult
- Various talks highlighting ways to tackle CoD

Beyond Single Neuron Dynamics

- Extension to large neuronal populations and network models
- Neural mass models approximate large-scale dynamics
- Opportunities to apply mean field theory and methods of finding approximate solutions to high-dimensional problems

Mitigating Curse of Dimensionality

- CoD arises as neuronal models increase in complexity
- Solving HJB equations in higher dimensions is difficult
- Various talks highlighting ways to tackle CoD

Reinforcement Learning for DBS

- OC is generally model-based and solving OC problems can be challenging
- RL is ideal when model is not available/accurate
- Opportunities to build virtual environments, cast control problem as a learning problem, and apply RL machinery

Some takeaways:

- Demonstrated the possibility of extending DL+OC to DBS
- Conventional (open-loop) DBS approaches can be improved
- Closed-loop DBS has high practical importance: for neurotech and patients
- Plenty of challenges and opportunities for researchers

Thank You!

Questions, comments, suggestions, collaborate?



malvern.madondo@emory.edu



[mmadondo.github.io](https://github.com/mmadondo)



- Alan L Hodgkin and Andrew F Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of physiology*, 117(4):500–544, 1952. URL <https://physoc.onlinelibrary.wiley.com/doi/pdfdirect/10.1113/jphysiol.1952.sp004764>.
- Derek Onken, Levon Nurbekyan, Xingjian Li, Samy Wu Fung, Stanley Osher, and Lars Ruthotto. A neural network approach for real-time high-dimensional optimal control, 2021.
- Lars Ruthotto, S J Osher, Wuchen Li, Levon Nurbekyan, and Samy Wu Fung. A machine learning framework for solving high-dimensional mean field game and mean field control problems. *Proceedings of the National Academy of Sciences*, 117(17):9183–9193, 2020.